MIT 8103 Advanced Database Systems

E-commerce Platform · Oluwapelumi Ejiyooye · Student ID 301877375

Case study

My case study is an E-commerce Platform. I designed the database for customers to search for products and make purchases. The store staff review customer orders and product sales with separate tables named customer, product, order_header and order_item. This supports these operations: create a product with a unique SKU; place an order containing one or more product items; review order details and summarize product sales.

Portfolio 1: Database Design and Modelling

I organised the relational design into 4 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. An order item must have a quantity more than zero. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 13 deliberately invalid inserts were rejected without changing the valid sample rows.

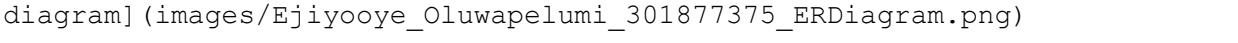


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

<!-- PAGE BREAK -->

Portfolio 2: Query Processing and Optimisation

I ran four SQL queries against the sample records. The join returned 1 rows and included alex@example.test with SKU-100. For the index comparison, I used 20,000 separate performance rows. The median lookup time changed from 1.041 ms to 0.185 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on product to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained unaccessible from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used order_events for the case's event documents. MongoDB inserted 2 sample events and returned the expected results for 4 queries. Validation

rejected an invalid date type and a duplicate event ID. Attached snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/Pelumieji/Ejiyooye_Oluwapelumi_301877375

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

<!-- PAGE BREAK -->

Portfolio 5: Distributed and Cloud Database Exercise

I recommended three MongoDB replica members across separate failure zones. The covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for order_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I stored customer, product, order_header, order_item in separate tables so their individual details could be maintained without repeating them in every related record. An order item should have a quantity greater than zero. The limitation tests check this alongside the required values and record links. One issue analysed by the two-session exercise is that both sessions can read the same value before either update is committed. Ensuring the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- [PostgreSQL: constraints](https://www.postgresql.org/docs/17/ddl-constraints.html)
- [PostgreSQL: EXPLAIN](https://www.postgresql.org/docs/17/using-explain.html)
- [PostgreSQL: isolation](https://www.postgresql.org/docs/17/transaction-iso.html)

- [PostgreSQL: row locks](https://www.postgresql.org/docs/17/explicit-locking.html)
- [MongoDB: data modelling](https://www.mongodb.com/docs/manual/data-modeling/)
- [MongoDB: replication](https://www.mongodb.com/docs/manual/replication/)
- [MongoDB: sharding](https://www.mongodb.com/docs/manual/sharding/)
- [Mermaid: ER diagrams](https://mermaid.js.org/syntax/entityRelationshipDiagram.html)